

Chapter 4. Introducing streams

This chapter covers

- What is a stream?
- Collections versus streams
- Internal versus external iteration
- Intermediate versus terminal operations

What would you do without collections in Java? Nearly every Java application *makes* and *processes* collections. Collections are fundamental to many programming tasks: they let you group and process data. To illustrate collections in action, imagine you are tasked to create a collection of dishes to represent a menu to calculate different queries. For example, you may want to find out the total number of calories for the menu. Or, you may need to filter the menu to select only low-calorie dishes for a special healthy menu. But despite collections being necessary for almost any Java application, manipulating collections is far from perfect:

- Much business logic entails database-like operations such as *grouping* a list of dishes by category (for example, all vegetarian dishes) or *finding* the most expensive dish. How many times do you find yourself re-implementing these operations using iterators? Most databases let you specify such operations declaratively. For example, the following SQL query lets you select (or “filter”) the names of dishes that are low in calories: `SELECT name FROM dishes WHERE calorie < 400`. As you can see, in SQL you don’t need to implement *how* to filter using the `calorie` attribute of a dish (as you would with Java collections, for example, using an iterator and an accumulator). Instead, you write *what* you want as result. This basic idea means that you worry less about how to explicitly implement such queries—it’s handled for you! Why can’t you do something similar with collections?
- How would you process a large collection of elements? To gain performance you’d need to process it in parallel and use multicore architec-

Other formats available



Audiobook



code is complicated in comparison to working on, it’s no fun to debug!

what could the Java language designers do to save your precious time and make your life as programmers easier? You may have guessed: the answer is *streams*.

4.1. What are streams?

Streams are an update to the Java API that let you manipulate collections of data in a declarative way (you express a query rather than code an ad hoc implementation for it). For now you can think of them as fancy iterators over a collection of data. In addition, streams can be processed in parallel *transparently*, without you having to write any multithreaded code! We explain in detail in [chapter 7](#) how streams and parallelization work. To see the benefits of using streams, compare the following code to return the names of dishes that are low in calories, sorted by number of calories—first in Java 7 and then in Java 8 using streams. Don’t worry about the Java 8 code too much; we explain it in detail in the next sections!

Before (Java 7):

```
List<Dish> lowCaloricDishes = new ArrayList<>();
for(Dish dish: menu) {
    if(dish.getCalories() < 400) {
        lowCaloricDishes.add(dish);
    }
}
Collections.sort(lowCaloricDishes, new Comparator<Dish>() {
    public int compare(Dish dish1, Dish dish2) {
        return Integer.compare(dish1.getCalories(), dish2.getCalories());
    }
});
List<String> lowCaloricDishesName = new ArrayList<>();
for(Dish dish: lowCaloricDishes) {
    lowCaloricDishesName.add(dish.getName());
}
```

- **1 Filters the elements using an accumulator**
- **2 Sorts the dishes with an anonymous class**
- **3 Processes the sorted list to select the names of dishes**

In this code you use a “garbage variable,” `lowCaloricDishes`. Its only purpose is to act as an intermediate throwaway container. In Java 8, this implementation detail is pushed into the library where it belongs.

Other formats available



```
import java.util.Comparator.comparing;
import static java.util.stream.Collectors.toList;
List<String> lowCaloricDishesName =
    menu.stream()
        .filter(d -> d.getCalories() < 400)
        .sorted(comparing(Dish::getCalories))
```

```
.map(Dish::getName)
.collect(toList());
```

3
4

- 1 Selects dishes that are below 400 calories
- 2 Sorts them by calories
- 3 Extracts the names of these dishes
- 4 Stores all the names in a List

To exploit a multicore architecture and execute this code in parallel, you need only to change `stream()` to `parallelStream()`:

```
List<String> lowCaloricDishesName =
    menu.parallelStream()
        .filter(d -> d.getCalories() < 400)
        .sorted(comparing(Dishes::getCalories))
        .map(Dish::getName)
        .collect(toList());
```

You may be wondering what exactly happens when you invoke the method `parallelStream`. How many threads are being used? What are the performance benefits? Should you use this method at all? [Chapter 7](#) covers these questions in detail. For now, you can see that the new approach offers several immediate benefits from a software engineering point of view:

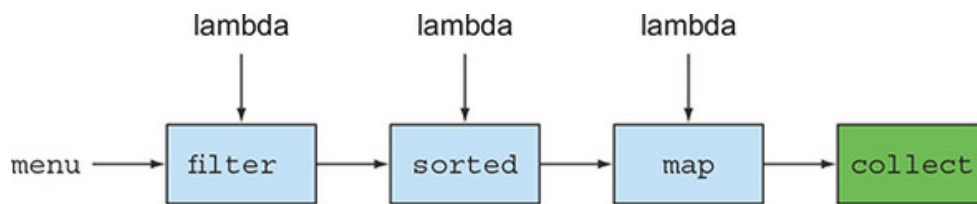
- The code is written in a *declarative way*: you specify *what* you want to achieve (*filter* dishes that are *low* in calories) as opposed to specifying *how* to implement an operation (using control-flow blocks such as loops and `if` conditions). As you saw in the previous chapter, this approach, together with behavior parameterization, enables you to cope with changing requirements: you could easily create an additional version of your code to filter high-calorie dishes using a lambda expression, without having to copy and paste code. Another way to think about the benefit of this approach is that the threading model is decoupled from the query itself. Because you are providing a recipe for a query, it could be executed sequentially or in parallel. You will learn more about this in [chapter 7](#).
- You chain together several building-block operations to express a com-

Other formats available



× pipeline (you chain the `filter` by linking operations, as illustrated in [figure 4.1](#)) readable and its intent clear. The result of the `filter` is passed to the `sorted` method, which is then passed to the `map` method and then to the `collect` method.

Figure 4.1. Chaining stream operations forming a stream pipeline



Because operations such as `filter` (or `sorted`, `map`, and `collect`) are available as *high-level building blocks* that don't depend on a specific threading model, their internal implementation could be single-threaded or could potentially maximize your multicore architecture transparently! In practice, this means you no longer have to worry about threads and locks to figure out how to parallelize certain data processing tasks: the Streams API does it for you!

The new Streams API is expressive. For example, after reading this chapter and [chapters 5](#) and [6](#), you'll be able to write code like the following:

```
Map<Dish.Type, List<Dish>> dishesByType =
    menu.stream().collect(groupingBy(Dish::getType));
```

This particular example is explained in detail in [chapter 6](#). It groups dishes by their types inside a `Map`. For example, the `Map` may contain the following result:

```
{FISH=[prawns, salmon],
 OTHER=[french fries, rice, season fruit, pizza],
 MEAT=[pork, beef, chicken]}
```

Now consider how you'd implement this with the typical imperative programming approach using loops. But don't waste too much of your time. Instead, embrace the power of streams in this and the following chapters!

Other libraries: Guava, Apache, and `lambdaj`

There have been many attempts at providing Java programmers with better libraries to manipulate collections. For example, Guava is a popular library created by Google. It provides additional container classes such as `multimap` and `multiset`. The Apache Commons Collections library pro-

Other formats available



`lambdaj`, written by Mario Fusco, coauthor of `lambdaj`, provides additional abilities to manipulate collections in a declarative style. `lambdaj` provides additional abilities to manipulate collections in a declarative style.

Now, Java 8 comes with its own official library for manipulating collections in a more declarative style.

To summarize, the Streams API in Java 8 lets you write code that's

- **Declarative**— More concise and readable
- **Composable**— Greater flexibility
- **Parallelizable**— Better performance

For the remainder of this chapter and the next, we'll use the following domain for our examples: a menu that's nothing more than a list of dishes

```
List<Dish> menu = Arrays.asList(
    new Dish("pork", false, 800, Dish.Type.MEAT),
    new Dish("beef", false, 700, Dish.Type.MEAT),
    new Dish("chicken", false, 400, Dish.Type.MEAT),
    new Dish("french fries", true, 530, Dish.Type.OTHER),
    new Dish("rice", true, 350, Dish.Type.OTHER),
    new Dish("season fruit", true, 120, Dish.Type.OTHER),
    new Dish("pizza", true, 550, Dish.Type.OTHER),
    new Dish("prawns", false, 300, Dish.Type.FISH),
    new Dish("salmon", false, 450, Dish.Type.FISH) );
```

where a Dish is an immutable class defined as

```
public class Dish {
    private final String name;
    private final boolean vegetarian;
    private final int calories;
    private final Type type;
    public Dish(String name, boolean vegetarian, int calories, Type type) {
        this.name = name;
        this.vegetarian = vegetarian;
        this.calories = calories;
        this.type = type;
    }
    public String getName() {
        return name;
    }
    public boolean isVegetarian() {
        return vegetarian;
    }
    public int getCalories() {
        return calories;
    }
    public Type getType() {
        return type;
    }
    @Override
    public String toString() {
        return name;
    }
}
```

Other formats available



Audiobook



```

    }
    public enum Type { MEAT, FISH, OTHER }
}

```

We'll now explore how you can use the Streams API in more detail. We'll compare streams to collections and provide some background. In the next chapter, we'll investigate in detail the stream operations available to express sophisticated data processing queries. We'll look at many patterns such as filtering, slicing, finding, matching, mapping, and reducing. There will be many quizzes and exercises to try to solidify your understanding.

Next, we'll discuss how you can create and manipulate numeric streams (for example, to generate a stream of even numbers or Pythagorean triples). Finally, we'll discuss how you can create streams from different sources, such as from a file. We'll also discuss how to generate streams with an infinite number of elements—something you definitely can't do with collections!

4.2. Getting started with streams

We start our discussion of streams with collections, because that's the simplest way to begin working with streams. Collections in Java 8 support a new `stream` method that returns a stream (the interface definition is available in `java.util.stream.Stream`). You'll later see that you can also get streams in various other ways (for example, generating stream elements from a numeric range or from I/O resources).

First, what exactly is a *stream*? A short definition is “a sequence of elements from a source that supports data-processing operations.” Let's break down this definition step-by-step:

- **Sequence of elements**— Like a collection, a stream provides an interface to a sequenced set of values of a specific element type. Because collections are data structures, they're mostly about storing and accessing elements with specific time/space complexities (for example, an `ArrayList` versus a `LinkedList`). But streams are about expressing computations such as `filter`, `sorted`, and `map`, which you saw earlier. Collections are about data; streams are about computations.

We explain this idea in greater detail in the coming sections.

Other formats available



Audiobook



from a data-providing source such as collections. Note that generating a stream from a collection preserves the ordering. The elements of a

stream coming from a list will have the same order as the list.

- **Data-processing operations**— Streams support database-like operations and common operations from functional programming languages to manipulate data, such as `filter`, `map`, `reduce`, `find`, `match`,

sort, and so on. Stream operations can be executed either sequentially or in parallel.

In addition, stream operations have two important characteristics:

- **Pipelining**— Many stream operations return a stream themselves, allowing operations to be chained to form a larger pipeline. This enables certain optimizations that we explain in the next chapter, such as *laziness* and *short-circuiting*. A pipeline of operations can be viewed as a database-like query on the data source.
- **Internal iteration**— In contrast to collections, which are iterated explicitly using an iterator, stream operations do the iteration behind the scenes for you. We briefly mentioned this idea in [chapter 1](#) and will return to it later in the next section.

Let's look at a code example to explain all of these ideas:

```
import static java.util.stream.Collectors.toList;
List<String> threeHighCaloricDishNames =
    menu.stream()                                1
        .filter(dish -> dish.getCalories() > 300) 2
        .map(Dish::getName)                       3
        .limit(3)                                  4
        .collect(toList());                       5
System.out.println(threeHighCaloricDishNames);    6
```

- **1 Gets a stream from menu (the list of dishes)**
- **2 Creates a pipeline of operations: first filter high-calorie dishes**
- **3 Gets the names of the dishes**
- **4 Selects only the first three**
- **5 Stores the results in another List**
- **6 Gives results [pork, beef, chicken]**

In this example, you first get a stream from the list of dishes by calling the `stream` method on `menu`. The *data source* is the list of dishes (the menu) and it provides *a sequence of elements* to the stream. Next, you apply a series of *data-processing operations* on the stream: `filter`, `map`, `limit`, and `collect`. All these operations except `collect` return another stream so they can be connected to form a *pipeline*, which can be viewed as a query

Other formats available

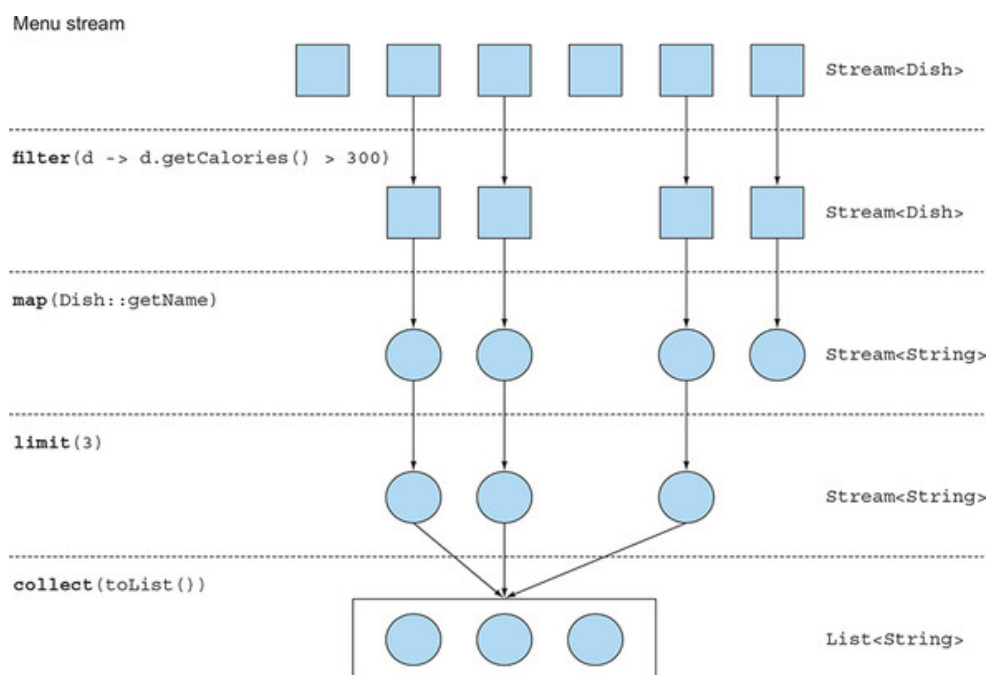


select operation starts processing the pipeline
different because it returns something other
No result is produced, and indeed no element

from menu is even selected, until `collect` is invoked. You can think of it as if the method invocations in the chain are queued up until `collect` is called. [Figure 4.2](#) shows the sequence of stream operations: `filter`, `map`, `limit`, and `collect`, each of which is briefly described here:

- **filter**— Takes a lambda to exclude certain elements from the stream. In this case, you select dishes that have more than 300 calories by passing the lambda `d -> d.getCalories() > 300`.
- **map**— Takes a lambda to transform an element into another one or to extract information. In this case, you extract the name for each dish by passing the method reference `Dish::getName`, which is equivalent to the lambda `d -> d.getName()`.
- **limit**— Truncates a stream to contain no more than a given number of elements.
- **collect**— Converts a stream into another form. In this case you convert the stream into a list. It looks like a bit of magic; we'll describe how `collect` works in more detail in [chapter 6](#). At the moment, you can see `collect` as an operation that takes as an argument various recipes for accumulating the elements of a stream into a summary result. Here, `toList()` describes a recipe for converting a stream into a list.

Figure 4.2. Filtering a menu using a stream to find out three high-calorie dish names



Notice how the code we described is different than what you'd write if you were to process the list of menu items step-by-step. First, you use a much more declarative style to process the data in the menu where you *say what needs to be done* (“Find names of three high-calorie dishes.” You

Other formats available



Audiobook



filter), extracting (map), or truncating (limit) are available through the Streams library. As you'll see, this gives you more flexibility to decide how to optimize this pipeline. For example, the filtering, extracting, and truncating steps could be merged into a single pass and stop as soon as three dishes are found. We show an example to demonstrate that in the next chapter.

Let's stand back a little and examine the conceptual differences between the Collections API and the new Streams API before we explore in more detail what operations you can perform with a stream.

4.3. Streams vs. collections

Both the existing Java notion of collections and the new notion of streams provide interfaces to data structures representing a sequenced set of values of the element type. By *sequenced*, we mean that we commonly step through the values in turn rather than randomly accessing them in any order. What's the difference?

We'll start with a visual metaphor. Consider a movie stored on a DVD. This is a collection (perhaps of bytes or of frames—we don't care which here) because it contains the whole data structure. Now consider watching the same video when it's being *streamed* over the internet. This is now a stream (of bytes or frames). The streaming video player needs to have downloaded only a few frames in advance of where the user is watching, so you can start displaying values from the beginning of the stream before most of the values in the stream have even been computed (consider streaming a live football game). Note particularly that the video player may lack the memory to buffer the whole stream in memory as a collection—and the startup time would be appalling if you had to wait for the final frame to appear before you could start showing the video. You might choose for video-player implementation reasons to *buffer* a part of a stream into a collection, but this is distinct from the conceptual difference.

In coarsest terms, the difference between collections and streams has to do with *when* things are computed. A collection is an in-memory data structure that holds *all* the values the data structure currently has—every element in the collection has to be computed before it can be added to the collection. (You can add things to, and remove them from, the collection, but at each moment in time, every element in the collection is stored in memory; elements have to be computed before becoming part of the collection.)

By contrast, a stream is a conceptually fixed data structure (you can't add or remove elements from it) whose elements are *computed on demand*.

Other formats available



✕ programming benefits. In [chapter 6](#), we'll construct a stream containing all the prime numbers, though there are an infinite number of them.

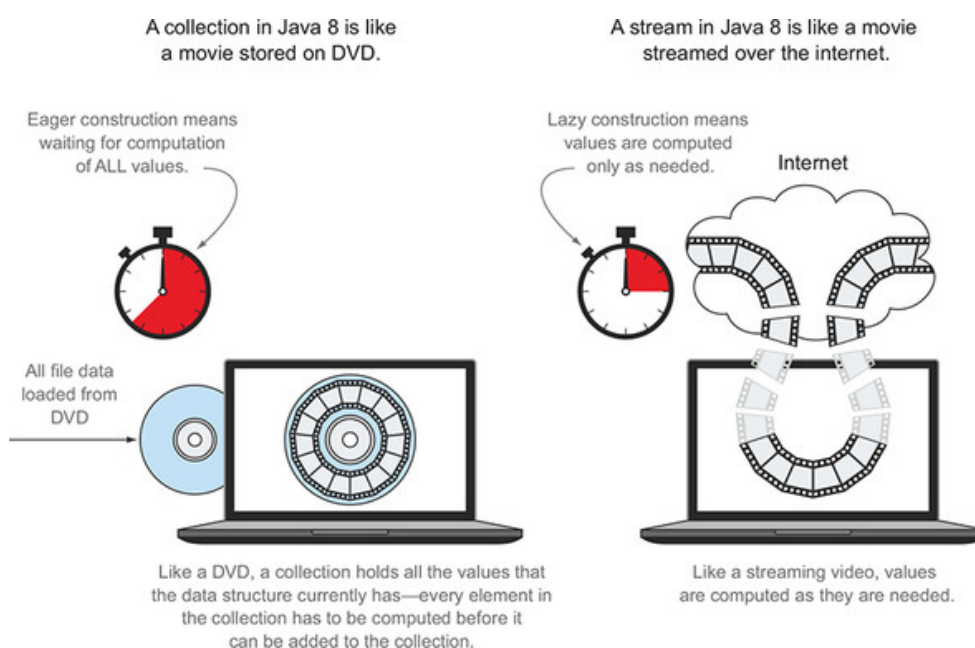
The idea is that a user will extract only the values they require from a stream and these elements are produced—invisibly to the user—only *as* and *when* required. This is a form of a producer-consumer relationship. Another view is that a stream is like a lazily constructed collection: values

are computed when they're solicited by a consumer (in management speak this is demand-driven, or even just-in-time, manufacturing).

In contrast, a collection is eagerly constructed (supplier-driven: fill your warehouse before you start selling, like a Christmas novelty that has a limited life). Imagine applying this to the primes example. Attempting to construct a collection of all prime numbers would result in a program loop that forever computes a new prime—adding it to the collection—but could never finish making the collection, so the consumer would never get to see it.

Figure 4.3 illustrates the difference between a stream and a collection, applied to our DVD versus internet streaming example.

Figure 4.3. Streams versus collections



Another example is a browser internet search. Suppose you search for a phrase with many matches in Google or in an e-commerce online shop. Instead of waiting for the whole collection of results along with their photographs to be downloaded, you get a stream whose elements are the best 10 or 20 matches, along with a button to click for the next 10 or 20. When you, the consumer, click for the next 10, the supplier computes these on demand, before returning them to your browser for display.

4.3.1. Traversable only once

Other formats available



Audiobook



..., a stream can be traversed only once. The stream is consumed. You can get a new stream from the initial data source to traverse it again as you would for an iterator (assuming it's a repeatable source like a collection; if it's an I/O channel, you're out of luck). For example, the following code would throw an exception indicating the stream has been consumed:

```
List<String> title = Arrays.asList("Modern", "Java", "In", "Action");
Stream<String> s = title.stream();
s.forEach(System.out::println);           1
s.forEach(System.out::println);           2
```

- **1 Prints each word in the title**
- **2 java.lang.IllegalStateException: stream has already been operated upon or closed**

Keep in mind that you can consume a stream only once!

Streams and collections philosophically

For readers who like philosophical viewpoints, you can see a stream as a set of values spread out in time. In contrast, a collection is a set of values spread out in space (here, computer memory), which all exist at a single point in time—and which you access using an iterator to access members inside a for–each loop.

Another key difference between collections and streams is how they manage the iteration over data.

4.3.2. External vs. internal iteration

Using the Collection interface requires iteration to be done by the user (for example, using for–each); this is called *external iteration*. The Streams library, by contrast, uses *internal iteration*—it does the iteration for you and takes care of storing the resulting stream value somewhere; you merely provide a function saying what’s to be done. The following code listings illustrate this difference.

Listing 4.1. Collections: external iteration with a for–each loop

```
List<String> names = new ArrayList<>();
for(Dish dish: menu) {
    names.add(dish.getName());
}
```

Other formats available



...st of menu sequentially

- **2 Extracts the name and adds it to an accumulator**

Note that the for–each hides some of the iteration complexity. The for–each construct is syntactic sugar that translates into something much

uglier using an Iterator object.

Listing 4.2. Collections: external iteration using an iterator behind the scenes

```
List<String> names = new ArrayList<>();
Iterator<String> iterator = menu.iterator();
while(iterator.hasNext()) {                                1
    Dish dish = iterator.next();
    names.add(dish.getName());
}
```

- **1 Iterates explicitly**

Listing 4.3. Streams: internal iteration

```
List<String> names = menu.stream()
                                .map(Dish::getName)          1
                                .collect(toList());           2
```

- **1 Parameterizes map with the getName method to extract the name of a dish**
- **2 Starts executing the pipeline of operations; no iteration**

Let's use an analogy to understand the differences and benefits of internal iteration. Let's say you're talking to your two-year-old daughter, Sofia, and want her to put her toys away:

- You: "Sofia, let's put the toys away. Is there a toy on the ground?"
- Sofia: "Yes, the ball."
- You: "Okay, put the ball in the box. Is there something else?"
- Sofia: "Yes, there's my doll."
- You: "Okay, put the doll in the box. Is there something else?"
- Sofia: "Yes, there's my book."
- You: "Okay, put the book in the box. Is there something else?"
- Sofia: "No, nothing else."
- You: "Fine, we're finished."

This is exactly what you do every day with your Java collections. You iter-

Other formats available

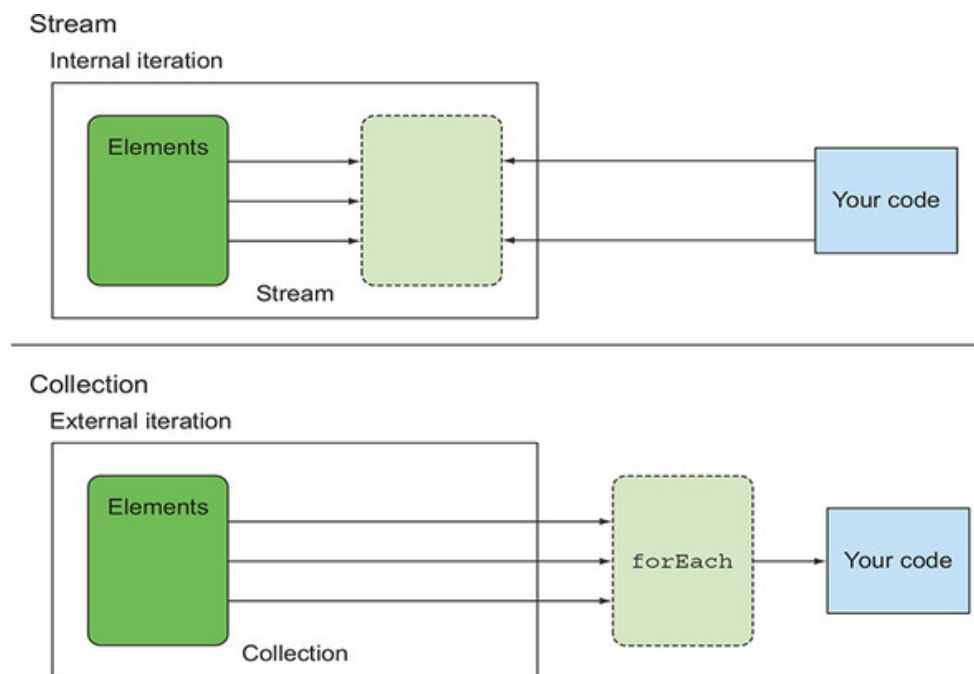


explicitly pulling out and processing the items after if you could tell Sofia, "Put all the toys in the box." There are two other reasons why an

internal iteration is preferable: first, Sofia could choose to take the doll with one hand and the ball with the other at the same time, and second, she could decide to take the objects closest to the box first and then the others. In the same way, using an internal iteration, the processing of

items could be transparently done in parallel or in a different order that may be more optimized. These optimizations are difficult if you iterate the collection externally as you're used to doing in Java. This may seem like nit-picking, but it's much of the *raison-d'être* of Java 8's introduction of streams. The internal iteration in the Streams library can automatically choose a data representation and implementation of parallelism to match your hardware. By contrast, once you've chosen external iteration by writing `for-each`, then you've committed to self-manage any parallelism. (*Self-managing* in practice means either "one fine day we'll parallelize this" or "starting the long and arduous battle involving tasks and synchronized.") Java 8 needed an interface like `Collection` but without iterators, ergo `Stream`! **Figure 4.4** illustrates the difference between a stream (internal iteration) and a collection (external iteration).

Figure 4.4. Internal versus external iteration



We've described the conceptual differences between collections and streams. Specifically, streams make use of internal iteration, where a library takes care of iterating for you. But this is useful only if you have a list of predefined operations to work with (for example, `filter` or `map`) that hide the iteration. Most of these operations take lambda expressions as arguments so you can parameterize their behavior as we showed in the previous chapter. The Java language designers shipped the Streams API with an extensive list of operations you can use to express complicated

Other formats available

 Audiobook 

4.1 below.



We'll briefly look at this list of operations now and follow up with examples in the next chapter. To test your understanding of internal versus external iteration, try quiz

Quiz 4.1: External vs. internal iteration

Based on what you learned about external iteration in [listing 4.1](#) and [4.2](#), which stream operation would you use to refactor the following code?

```
List<String> highCaloricDishes = new ArrayList<>();
Iterator<String> iterator = menu.iterator();
while(iterator.hasNext()) {
    Dish dish = iterator.next();
    if(dish.getCalories() > 300) {
        highCaloricDishes.add(d.getName());
    }
}
```

Answer: You need to use the filter pattern

```
List<String> highCaloricDish =
    menu.stream()
        .filter(dish -> dish.getCalories() > 300)
        .collect(toList());
```

Don't worry if you're still unfamiliar with how to precisely write a stream query, you will learn this in more detail in the next chapter.

4.4. Stream operations

The streams interface in `java.util.stream.Stream` defines many operations. They can be classified into two categories. Let's look at our previous example once again:

```
List<String> names = menu.stream()           1
    .filter(dish -> dish.getCalories() > 300) 2
    .map(Dish::getName)                       2
    .limit(3)                                 2
    .collect(toList());                       3
```

- **1 Gets a stream from the list of dishes**

- **2 Intermediate operation**

Other formats available



Audiobook



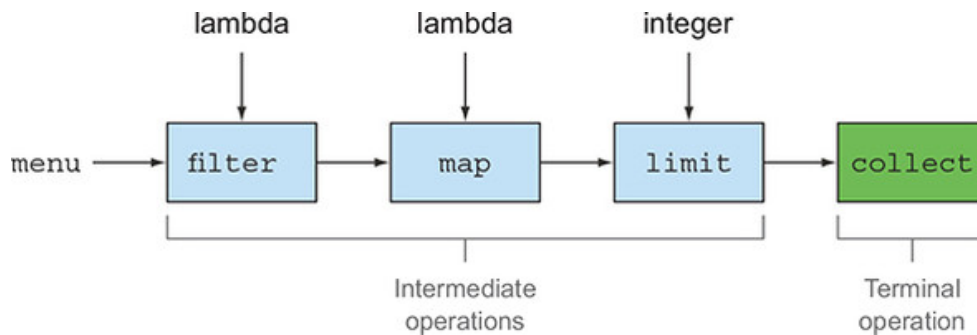
to a List

ations:

- `filter`, `map`, and `limit` can be connected together to form a pipeline.
- `collect` causes the pipeline to be executed and closes it.

Stream operations that can be connected are called *intermediate operations*, and operations that close a stream are called *terminal operations*. **Figure 4.5** highlights these two groups. Why is the distinction important?

Figure 4.5. Intermediate versus terminal operations



4.4.1. Intermediate operations

Intermediate operations such as `filter` or `sorted` return another stream as the return type. This allows the operations to be connected to form a query. What's important is that intermediate operations don't perform any processing until a terminal operation is invoked on the stream pipeline—they're lazy. This is because intermediate operations can usually be merged and processed into a single pass by the terminal operation.

To understand what's happening in the stream pipeline, modify the code so each lambda also prints the current dish it's processing. (Like many demonstration and debugging techniques, this is appalling programming style for production code, but directly explains the order of evaluation when you're learning.)

```
List<String> names =
    menu.stream()

        .filter(dish -> {
            System.out.println("filtering:" + dish.getName());
            return dish.getCalories() > 300;
        })
        .map(dish -> {
            System.out.println("mapping:" + dish.getName());
            return dish.getName();
        })
        .limit(2)
```

Other formats available



Audiobook



- 1 Prints the dishes as they're filtered
- 2 Prints the dishes as you extract their names

This code, when executed, will print the following:


```
filtering:pork
mapping:pork
filtering:beef
mapping:beef
filtering:chicken
mapping:chicken
[pork, beef, chicken]
```

By doing this, you can notice that the Streams library performs several optimizations exploiting the lazy nature of streams. First, despite the fact that many dishes have more than 300 calories, only the first three are selected! This is because of the `limit` operation and a technique called *short-circuiting*, as we'll explain in the next chapter. Second, despite the fact that `filter` and `map` are two separate operations, they were merged into the same pass (compiler experts call this technique *loop fusion*).

4.4.2. Terminal operations

Terminal operations produce a result from a stream pipeline. A result is any non-stream value such as a `List`, an `Integer`, or even `void`. For example, in the following pipeline, `forEach` is a terminal operation that returns `void` and applies a lambda to each dish in the source. Passing `System.out.println` to `forEach` asks it to print every `Dish` in the stream created from `menu`:

```
menu.stream().forEach(System.out::println);
```

To check your understanding of intermediate versus terminal operations, try out quiz 4.2.

Quiz 4.2: Intermediate vs. terminal operations

In the stream pipeline that follows, can you identify the intermediate and terminal operations?

```
long count = menu.stream()
    .filter(dish -> dish.getCalories() > 300)
    .distinct()
    .limit(3)
    .count();
```

Other formats available



Answer:

The last operation in the stream pipeline count returns a long, which is a non-stream value. It's therefore a *terminal operation*. All previous operations, filter, distinct, limit, are connected and return a stream. They are therefore *intermediate operations*.

4.4.3. Working with streams

To summarize, working with streams in general involves three items:

- A *data source* (such as a collection) to perform a query on
- A chain of *intermediate operations* that form a stream pipeline
- A *terminal operation* that executes the stream pipeline and produces a result

The idea behind a stream pipeline is similar to the builder pattern (see [http://en.wikipedia.org/wiki/Builder pattern](http://en.wikipedia.org/wiki/Builder_pattern)). In the builder pattern, there's a chain of calls to set up a configuration (for streams this is a chain of intermediate operations), followed by a call to a build method (for streams this is a terminal operation).

For convenience, [tables 4.1](#) and [4.2](#) summarize the intermediate and terminal stream operations you've seen in the code examples so far. Note that this is an incomplete list of operations provided by the Streams API; you'll see several more in the next chapter!

Table 4.1. Intermediate operations

Operation	Type	Return type	Argument of the operation	Function descriptor
filter	Intermediate	Stream<T>	Predicate<T>	T -> boolean
map	Intermediate	Stream<R>	Function<T, R>	T -> R
limit	Intermediate	Stream<T>		
		Stream<T>	Comparator<T>	(T, T) -> int
distinct	Intermediate	Stream<T>		

Other formats available

 Audiobook 

Table 4.2. Terminal operations

Operation	Type	Return type	Purpose
forEach	Terminal	void	Consumes each element from a stream and applies a lambda to each of them.
count	Terminal	long	Returns the number of elements in a stream.
collect	Terminal	(generic)	Reduces the stream to create a collection such as a List, a Map, or even an Integer. See chapter 6 for more detail.

4.5. Road map

In the next chapter, we’ll detail the available stream operations with use cases so you can see what kinds of queries you can express with them. We look at many patterns such as filtering, slicing, finding, matching, mapping, and reducing, which can be used to express sophisticated data-processing queries.

[Chapter 6](#) then explores collectors in detail. In this chapter we have only made use of the `collect()` terminal operation on streams (see [table 4.2](#)) in the stylized form of `collect(toList())`, which creates a `List` whose elements are the same as those of the stream it’s applied to.

Summary

- A stream is a sequence of elements from a source that supports data-processing operations.
- Streams make use of internal iteration: the iteration is abstracted away through operations such as `filter`, `map`, and `sorted`.
- There are two types of stream operations: intermediate and terminal operations.
- Intermediate operations such as `filter` and `map` return a stream and can be chained together. They’re used to set up a pipeline of operations.
- Terminal operations such as `forEach` and `count` return a non-stream pipeline to return a result.
- The elements of a stream are computed on demand (“lazily”).

Other formats available



Other formats available



Audiobook

